

Codex Master Prompt

You are working directly inside the existing `AG_Foundational_Model` repository.

Your task is to complete all remaining software required for a publication-grade close-field agricultural vision foundation model research project.

Do not return a plan, pseudocode, mock implementation, or partial patch. Inspect the existing repository, preserve working behavior, and implement complete production-quality code.

The project must follow test-driven development.

1. Strict execution restrictions

Follow these rules without exception.

1. Do not inspect, enumerate, open, extract, hash, sample, or analyze any real project dataset.
2. Do not access `../Pertaining`, `../Pretraining`, external dataset folders, user home data directories, ZIP archives, GeoTIFF collections, or downloaded benchmark datasets.
3. Do not run any corpus audit or contamination scan on real data.
4. Do not run full model pretraining.
5. Do not run downstream training.
6. Do not run a complete benchmark.
7. Do not download pretrained weights.
8. Do not access the internet.
9. Do not use a GPU.
10. Do not run the complete test suite with commands such as `pytest`, `pytest -q`, or equivalent commands without explicit test paths.
11. Do not run the existing end-to-end demo training commands.
12. Do not modify or delete user data.
13. Do not produce fake research results or simulated benchmark scores.

You may do the following.

1. Inspect source code, configuration files, documentation, package metadata, and existing tests.
2. Create tiny synthetic images, masks, annotations, manifests, and tensors inside temporary test directories.
3. Use deterministic fake backbones and mocked distributed operations in tests.
4. Run only narrowly targeted unit tests related to the code currently being implemented.
5. Run Ruff or static checks only on modified files.
6. Perform one or two forward and backward steps using tiny test doubles inside unit tests.
7. Use `tmp_path`, monkeypatching, and in-memory fixtures.
8. Add optional integrations that remain disabled when their dependencies are unavailable.

2. TDD workflow

For every feature, follow this exact sequence.

1. Read the existing implementation and public interfaces.
2. Write focused tests that describe the required behavior.
3. Run only the new test file and confirm that the new tests initially fail for the expected reason.
4. Implement the smallest complete solution.
5. Run only the relevant tests until they pass.
6. Refactor without changing behavior.
7. Run the directly related existing tests.
8. Continue to the next feature.

Never write all production code first and add superficial tests later.

Tests must validate behavior, not merely increase coverage.

Do not weaken, delete, skip, or mark existing tests as expected failures to make the implementation pass.

3. Completion requirements

The final repository must satisfy these requirements.

1. No unfinished `TODO`, `FIXME`, `pass`, placeholder function, fake metric, or unimplemented CLI path.
2. No pseudocode in production modules.
3. No silent exception handling.
4. No hidden fallback from pretrained initialization to random initialization.
5. No dependency on an absolute local path.
6. No automatic access to real datasets during import or test collection.
7. No network access during tests.
8. No heavy model construction during unit tests.
9. All new public functions must have type hints and concise docstrings.
10. All new configuration values must be validated before model or data construction.
11. Existing MIM, DINO, adapters, checkpoints, logging, and resume behavior must remain backward compatible.
12. Newly added artifacts must use the existing atomic writing infrastructure.
13. Rank-zero-only writes must be enforced under distributed training.
14. Every new CLI command must support `--help` without importing optional heavy dependencies.
15. Every new experiment must support a dry-run configuration validation mode that performs no training and reads no image data.
16. Every benchmark result must map to a run ID, configuration, split hash, checkpoint hash, and prediction artifact.
17. Use precise naming. Do not label the implementation a complete DINOv3 reproduction. Use terms such as `DINOv3-style agricultural continual pretraining` where appropriate.

4. Preserve the current repository design

The repository already separates responsibilities approximately as follows.

```
src/ag_foundation/  
  cli.py  
  command_logging.py  
  
  data/  
    dataset.py  
    geotiff.py  
    demo.py  
  
  models/  
    official_vit.py  
    vit.py  
    mim.py  
    dino.py  
  
  training/  
    mim_runner.py  
    dino_runner.py  
    ssl_trainer.py  
    dino_trainer.py  
    state.py  
    artifacts.py  
    visualization.py  
    experiment_metadata.py
```

Preserve these boundaries.

Data modules own loading, manifests, catalogs, grouping, splitting, and sampling.

Model modules own adapters, backbones, heads, losses, and forward outputs.

Training modules own optimization, precision, distributed behavior, metrics, checkpointing, and resume.

Runner modules own configuration validation and object construction.

Benchmark modules own downstream datasets, task heads, adaptation protocols, evaluation, and result aggregation.

Do not create one oversized module containing unrelated responsibilities.

5. Begin with a codebase inventory

Before changing production code, inspect the repository and create an internal implementation checklist covering:

1. Existing CLI commands.
2. Existing configuration dataclasses.
3. Existing model interfaces.
4. Existing checkpoint schema.
5. Existing artifact and manifest writers.
6. Existing test doubles for `timm`.
7. Existing optional dependency patterns.
8. Existing distributed sampler support.
9. Existing DINO teacher update behavior.
10. Existing MIM masking implementation.
11. Existing test naming and fixture conventions.

Do not create a user-facing planning document before implementation.

Use the inventory only to avoid duplicating existing functionality.

Part A. Distributed and scalable training

6. Distributed runtime

Implement a reusable distributed runtime module.

Suggested location:

```
src/ag_foundation/training/distributed.py
```

It must support:

1. Initialization from `torchrun` environment variables.
2. Single-process CPU, MPS, and CUDA execution.
3. Multi-process CUDA distributed data parallel execution.
4. Rank, local rank, world size, and main-process helpers.
5. Configurable backend with safe defaults.
6. Process-group cleanup.
7. Barriers where required.
8. Tensor reduction.
9. Dictionary metric reduction.
10. Object gathering for small metadata.
11. Deterministic rank-specific seed derivation.

12. Rank-zero-only context helpers.
13. Helpful errors for invalid environment variables.
14. No-op behavior in ordinary single-process execution.

Add tests using mocks. Do not launch real distributed subprocesses.

7. Distributed launchers

Add supported launch paths.

```
scripts/train_mim_distributed.sh
scripts/train_dino_distributed.sh
scripts/train_joint_distributed.sh

scripts/train_mim_distributed.ps1
scripts/train_dino_distributed.ps1
scripts/train_joint_distributed.ps1
```

Also support direct commands such as:

```
torchrun --nproc-per-node 8 -m ag_foundation train-joint --config ...
```

Launchers must:

1. Preserve command logging.
2. Forward extra arguments.
3. Avoid duplicate logging from every rank.
4. Return the underlying process exit status.
5. Fail clearly when `torchrun` is unavailable.
6. Avoid machine-specific paths.

8. Distributed model wrapping

Update trainers so that:

1. The student model can be wrapped with DDP.
2. The teacher remains inference-only.
3. Model access works through a consistent unwrap helper.
4. EMA updates use unwrapped student parameters.
5. Batch normalization handling remains correct.
6. Metric reductions use global sample counts.
7. Only rank zero writes metrics, visualizations, checkpoints, and manifests.
8. Every rank restores its correct RNG state.
9. Checkpoints created under DDP load in single-process mode.
10. Single-process checkpoints load under DDP.

11. Resume does not duplicate an epoch.
12. Existing checkpoint migration remains functional.

9. Gradient accumulation

Add configurable gradient accumulation to all training paths.

Configuration:

```
runtime:  
  gradient_accumulation_steps: 1
```

Requirements:

1. Validate that the value is a positive integer.
2. Divide loss correctly before backward.
3. Step optimizer and scheduler only at accumulation boundaries.
4. Handle the final incomplete accumulation group.
5. Use DDP `no_sync` for non-step microbatches.
6. Apply gradient clipping immediately before optimizer step.
7. Update the EMA teacher only after optimizer steps.
8. Track microbatch count and optimizer-step count.
9. Save both counters in checkpoints.
10. Resume correctly from an epoch boundary.
11. Record local batch size, accumulation steps, world size, and effective global batch size in manifests.
12. Preserve current behavior when the value is one.

10. Step-based schedules

Add reusable schedule functions for:

1. Learning rate warmup and cosine decay.
2. Weight decay cosine scheduling.
3. EMA teacher momentum scheduling.
4. Teacher temperature warmup.
5. Loss-weight scheduling.
6. Morphology mask curriculum.
7. High-resolution stage transitions.

Schedules must operate by optimizer step rather than only by epoch.

Add tests for:

- First step.
- Warmup boundary.
- Midpoint.

- Last step.
- Resume offset.
- One-step schedule.
- Invalid total step counts.

Part B. Dataset mixture, manifests, and governance

11. Publication manifest schema

Add a strict publication data manifest.

Suggested modules:

```
src/ag_foundation/data/manifest.py
src/ag_foundation/data/schemas.py
```

Support CSV, JSON, and JSONL.

Required fields should include:

```
sample_id
path
source_id
source_name
license
license_url
version
download_date
checksum
perceptual_hash
duplicate_group
split
group
crop
species
disease
organ
country
region
farm
field
acquisition_date
```

```
season
sensor_id
sensor_model
channels
band_names
wavelengths_nm
spatial_resolution
quality_status
exclusion_reason
```

Not every field must contain a value, but every manifest row must follow the same validated schema.

Requirements:

1. Stable canonical serialization.
2. Deterministic manifest hashing.
3. Relative path support.
4. Archive member path support.
5. Validation of channel and wavelength list lengths.
6. Validation of split names.
7. Duplicate `sample_id` detection.
8. Clear row-level validation errors.
9. No file opening during schema validation unless explicitly requested.
10. Manifest summary generation from metadata only.
11. Manifest versioning.
12. Forward-compatible optional fields.

12. Source-balanced sampling

Implement:

1. Proportional source sampling.
2. Square-root source sampling.
3. Uniform source sampling.
4. User-defined source weights.
5. Dataset-balanced sampling.
6. Crop-balanced sampling.
7. Configurable mixture keys.
8. Minimum and maximum source probability clipping.
9. Deterministic epoch reseeding.
10. Distributed weighted sampling without duplicate rank behavior.
11. Correct handling of tiny sources.
12. Correct behavior when a source has one sample.
13. Effective sampling probability reports.

Suggested modules:


```
src/ag_foundation/data/samplers.py
src/ag_foundation/data/mixture.py
```

Configuration example:

```
data:
  sampling:
    strategy: power
    group_key: source_id
    alpha: 0.5
    replacement: true
    samples_per_epoch: null
```

13. Group-aware split registry

Implement a split system that can create and validate:

- Source-held-out splits.
- Farm-held-out splits.
- Field-held-out splits.
- Geography-held-out splits.
- Season-held-out splits.
- Sensor-held-out splits.
- Crop-held-out splits.
- Observation-held-out splits.
- User-supplied fixed splits.

Suggested module:

```
src/ag_foundation/data/splits.py
```

Requirements:

1. Deterministic splitting.
2. No group overlap.
3. Configurable validation and test proportions.
4. Stratification where valid.
5. Clear reporting when exact proportions are impossible.
6. Split manifest hashing.
7. Leakage validation.
8. Split comparison utilities.
9. No inspection of image pixels.
10. Synthetic metadata tests.

14. Deduplication and contamination code

Implement code only. Do not run it on real data.

Suggested package:

```
src/ag_foundation/data/governance/  
    exact_hash.py  
    perceptual_hash.py  
    embedding_index.py  
    duplicate_clusters.py  
    contamination.py  
    reports.py
```

Capabilities:

1. Streamed SHA256 hashing.
2. Optional perceptual hashing.
3. Duplicate clustering.
4. Cross-manifest exact matching.
5. Cross-manifest perceptual matching.
6. Optional embedding-neighbor matching from precomputed embeddings.
7. Crop-aware matching when precomputed crop hashes are supplied.
8. Threshold configuration.
9. Exclusion manifest generation.
10. Human-review queue generation.
11. Deterministic report output.
12. Dry-run mode.
13. Resume from partial metadata.
14. No automatic deletion.
15. No image embedding computation unless explicitly invoked by the user.

Tests must use tiny generated images in temporary directories.

Part C. Publication-grade SSL objectives

15. iBOT-style patch objective

Add a patch-level teacher-student objective.

Suggested modules:

```
src/ag_foundation/models/ibot.py
src/ag_foundation/models/heads.py
```

Requirements:

1. Masked student patch prediction.
2. Unmasked teacher patch targets.
3. Separate patch projection head.
4. Teacher centering for patch outputs.
5. Configurable output dimension.
6. Configurable student and teacher temperatures.
7. Support for multiple global crops.
8. Correct cross-view pairing.
9. Mask-aware loss reduction.
10. Stable behavior when a sample has no valid masked patches.
11. EMA teacher updates for the patch head.
12. Complete checkpoint serialization.
13. Backward-compatible loading of old DINO checkpoints.
14. Numerically stable softmax and log-softmax.
15. Finite-value validation.

Tests must use tiny token tensors and deterministic fake encoders.

16. Morphology-aware mask generators

Implement a mask framework.

Suggested package:

```
src/ag_foundation/models/masking/
  base.py
  random_block.py
  foreground.py
  boundary.py
  mixed.py
  curriculum.py
```

Implement these mask types.

Random block mask

Standard spatial block masking with configurable ratio and block geometry.

Foreground-concentrated mask

Generate a plant-likelihood map using one of these inputs:

1. A supplied attention map.
2. A supplied saliency map.
3. A deterministic image-based fallback using saturation, excess green, and edge evidence.

Do not require labels.

Boundary-detail mask

Use differentiable or tensor-compatible gradient operations to emphasize:

- Leaf boundaries.
- Thin structures.
- Lesion edges.
- High-frequency plant detail.

Mixed morphology mask

Sample among random, foreground, and boundary masks using configured probabilities.

Curriculum mask

Change mask probabilities and mask ratios according to optimizer-step progress.

Requirements:

1. Deterministic output under a fixed generator.
2. Exact requested number of masked patches where possible.
3. Batch support.
4. Multiple image resolutions.
5. Patch-size awareness.
6. CPU and CUDA compatibility.
7. No NumPy round-trip inside forward paths.
8. No gradient requirement for mask selection.
9. Safe behavior for uniform images.
10. Safe behavior for all-black and all-white images.
11. Visualization utilities for debugging.
12. Configuration validation.

17. Morphology reconstruction objective

Extend MIM so that it can use any mask generator.

Support reconstruction targets:

1. Adapted RGB-like pixels.
2. Normalized patch pixels.
3. Teacher patch features.
4. A weighted combination of feature and pixel targets.

Implement configurable per-patch weighting from foreground and boundary scores.

Tests must confirm:

- Only masked patches contribute.
- Weight normalization is correct.
- Zero-weight patches do not produce NaN.
- Feature target dimensions are validated.
- Existing random MIM behavior remains unchanged by default.

18. Gram anchoring

Implement dense feature Gram anchoring.

Suggested module:

```
src/ag_foundation/models/gram_anchor.py
```

Requirements:

1. Frozen reference encoder support.
2. Optional EMA reference encoder support.
3. Selected transformer layer hooks.
4. Patch-token extraction without CLS or register tokens.
5. Token normalization.
6. Gram matrix computation.
7. Configurable layer weights.
8. Configurable spatial subsampling for memory control.
9. Mixed precision safety.
10. No gradients through the reference encoder.
11. Reference model state in checkpoints when required.
12. Helpful errors when selected layer names do not exist.
13. Compatibility with ViT S, B, and L variants.
14. Tests using fake transformer blocks.

Do not load a real DINOv3 checkpoint in tests.

19. Joint SSL model

Create a joint agricultural SSL model that combines:

```
DINO global loss
iBOT patch loss
morphology-aware masked loss
Gram anchoring loss
```

Suggested modules:

```
src/ag_foundation/models/joint_ssl.py
src/ag_foundation/training/joint_runner.py
src/ag_foundation/training/joint_trainer.py
```

The forward result must be a typed structure containing:

```
total_loss
dino_loss
ibot_loss
morphology_loss
gram_loss
student_entropy
teacher_entropy
masked_patch_count
valid_sample_count
```

Requirements:

1. Each loss can be enabled or disabled independently.
2. Loss weights can be constant or scheduled.
3. Disabled objectives construct no unnecessary heavy modules.
4. Old MIM and DINO CLI paths remain usable.
5. Shared backbone computation should be reused where practical.
6. Teacher updates happen only after optimizer steps.
7. All objective states enter checkpoints.
8. Metrics enter CSV and JSON artifacts.
9. Resume restores every objective state.
10. Configuration combinations are validated early.
11. No loss may silently return zero because of a shape mismatch.
12. Model outputs must remain finite.

20. Sequential pretraining

Implement a stage runner for:

1. MIM then DINO.
2. MIM then joint SSL.
3. DINO then high-resolution refinement.
4. Joint SSL then Gram anchoring refinement.

Suggested files:

```
src/ag_foundation/training/stages.py
src/ag_foundation/training/pipeline_runner.py
```

A pipeline YAML should support:

```
pipeline:
  stages:
    - name: mim
      config: configs/...
    - name: joint
      config: configs/...
      initialize_from: previous
    - name: high_resolution
      config: configs/...
      initialize_from: previous
```

Requirements:

1. Strict stage ordering.
2. Compatibility validation.
3. Transfer of backbone and adapter weights.
4. Optional transfer of optimizer state.
5. Explicit reset of incompatible heads.
6. Immutable stage manifests.
7. Stage checkpoint hashes.
8. Resume from a completed or interrupted stage.
9. Dry-run validation.
10. No training when dry-run is enabled.

21. High-resolution adaptation

Implement a high-resolution stage that:

1. Interpolates positional embeddings.

2. Validates patch-size divisibility.
3. Preserves checkpoint compatibility.
4. Supports reduced learning rates.
5. Supports Gram anchoring.
6. Records old and new token grid sizes.
7. Avoids reinitializing pretrained layers.
8. Includes synthetic shape tests for several resolutions.

Part D. Configuration and CLI

22. Typed configuration

Extend the existing strict YAML system.

Add sections for:

```
distributed:
sampling:
ibot:
masking:
morphology:
gram_anchor:
joint_ssl:
pipeline:
benchmark:
adaptation:
evaluation:
statistics:
robustness:
compute_tracking:
```

Requirements:

1. Reject unknown keys.
2. Validate mutually exclusive settings.
3. Resolve paths relative to the YAML file.
4. Preserve CLI override behavior.
5. Support serialization to resolved YAML.
6. Avoid importing heavy optional dependencies during parsing.
7. Add unit tests for valid and invalid combinations.
8. Add migration logic for existing configs.

23. New CLI commands

Implement these commands.


```
train-joint
train-pipeline
validate-config
build-manifest
validate-manifest
build-splits
validate-splits
find-duplicates
check-contamination
benchmark
evaluate-predictions
aggregate-results
generate-experiment-matrix
export-repro-bundle
```

Each command must:

1. Have useful `--help`.
2. Support explicit output paths.
3. Support dry-run where relevant.
4. Use command logging.
5. Produce structured machine-readable output.
6. Fail with actionable errors.
7. Avoid touching real data unless the user explicitly invokes it.
8. Remain importable without optional benchmark dependencies.

Part E. Downstream benchmark framework

24. Benchmark package structure

Create a coherent package.

```
src/ag_foundation/benchmark/
  registry.py
  schemas.py
  datasets/
  models/
  heads/
  adaptation/
  tasks/
  metrics/
  robustness/
  statistics/
```

```
runners/  
artifacts/
```

25. Dataset registry

Implement declarative dataset specifications.

A dataset specification must describe:

```
name  
version  
task  
root  
manifest  
train_split  
validation_split  
test_split  
class_names  
group_columns  
input_channels  
annotation_format  
license  
citation  
strict_pretraining_holdout
```

Support task types:

```
classification  
multilabel_classification  
object_detection  
semantic_segmentation  
instance_segmentation  
count_regression  
density_estimation  
retrieval  
open_set_classification
```

The registry must not download datasets.

It should only resolve configured local paths when a command is explicitly run.

26. Generic dataset adapters

Implement reusable dataset readers.

Classification

Support:

- Class folders.
- CSV manifests.
- Multilabel columns.
- Hierarchical labels.

Object detection

Support:

- COCO JSON.
- A generic CSV box format.

Semantic segmentation

Support:

- Image and mask pairs.
- Palette masks.
- Integer class masks.

Instance segmentation

Support COCO polygon and RLE annotations through an optional `pycocotools` dependency.

Counting

Support:

- Scalar counts in CSV.
- Point annotations.
- Density maps.

Retrieval

Support:

- Class-based retrieval.
- Query and gallery manifests.
- Cross-domain query and gallery splits.

Requirements:

1. No dataset-specific hard-coded user path.
2. Clear validation errors.

3. Deterministic transforms.
4. Synthetic fixture tests.
5. Group metadata propagation.
6. Sample ID propagation into predictions.
7. Optional metadata output.

27. Foundation encoder interface

Create a shared encoder wrapper that can load:

1. Current project checkpoints.
2. ImageNet `timm` backbones.
3. MAE backbones.
4. DINOv2 backbones.
5. DINOv3 backbones.
6. Randomly initialized official backbones.

Support:

- CLS output.
- Mean patch pooling.
- CLS plus mean pooling.
- Selected intermediate layers.
- Frozen mode.
- Full fine-tuning mode.
- Adapter-only mode.
- LoRA mode.
- Decoder-only mode.
- Input-channel adapter control.
- Checkpoint key migration.

Do not download weights in tests.

28. LoRA

Implement LoRA for ViT attention and optionally MLP projections.

Requirements:

1. Configurable rank.
2. Configurable alpha.
3. Configurable dropout.
4. Target module selection.
5. Merge and unmerge.
6. Trainable parameter reporting.
7. Checkpoint save and load.
8. No modification of unsupported layers.

9. Tests proving the frozen base weights do not change.
10. Rank 4, 8, and 16 config templates.

29. Classification heads and runners

Implement:

1. Linear probe.
2. Attentive probe.
3. Full fine-tuning classifier.
4. Hierarchical classifier where taxonomy exists.
5. Multilabel classifier.

Support:

- Top-1.
- Top-5.
- Macro F1.
- Balanced accuracy.
- Mean class recall.
- Per-class metrics.
- Confusion matrices.
- Calibration.

30. Frozen dense probe

Implement a lightweight dense evaluation path.

Requirements:

1. Frozen patch tokens.
2. Final-layer features.
3. Optional last-four-layer concatenation.
4. Linear pixel classifier.
5. Bilinear upsampling.
6. Ignore-index support.
7. mIoU.
8. Per-class IoU.
9. Boundary F1.
10. Validation checkpointing.

31. Semantic segmentation

Implement a UPerNet-style decoder compatible with ViT intermediate features.

Requirements:

1. Configurable feature layers.
2. Pyramid projection.
3. Pyramid pooling.
4. Feature fusion.
5. Auxiliary loss option.
6. Ignore-index support.
7. Variable class count.
8. Full fine-tuning and frozen encoder modes.
9. Synthetic mask tests.
10. No dependency on a specific agricultural dataset.

32. Detection backbone adapter

Implement a ViT feature pyramid adapter suitable for torchvision detection models.

Support:

1. Faster R-CNN.
2. Mask R-CNN.
3. Shared backbone output dictionary.
4. Configurable selected transformer layers.
5. Configurable output channels.
6. Frozen backbone.
7. LoRA backbone.
8. Full fine-tuning.
9. Correct image-size propagation.
10. Tiny synthetic forward tests using mocked features.

Do not run full detection training.

33. Counting

Implement:

1. CLS regression head.
2. Patch-token density head.
3. Detection-derived counting evaluator.

Metrics:

- MAE.
- RMSE.
- MAPE with safe zero handling.
- R-squared.
- Bias.

- Per-domain MAE.

34. Retrieval

Implement:

1. Embedding extraction.
2. L2 normalization.
3. Cosine retrieval.
4. Query-gallery mode.
5. Class-based mode.
6. Cross-source mode.
7. Chunked similarity computation.
8. Recall at K.
9. mAP at K.
10. NMI.
11. Adjusted Rand index.
12. Exclusion of self matches.
13. Deterministic result ordering.

35. Open-set recognition

Implement evaluation methods:

1. Maximum softmax probability.
2. Energy score.
3. Prototype distance.
4. k-nearest-neighbor distance.
5. Mahalanobis distance.

Metrics:

- AUROC.
- AUPR.
- FPR95.
- Open-set classification rate.
- Accuracy at fixed coverage.

Use validation data to select thresholds. Never select thresholds on the test split.

Part F. Evaluation and statistical analysis

36. Unified prediction format

Define a versioned prediction schema containing:

```
run_id
checkpoint_hash
dataset
split
sample_id
group_id
task
target
prediction
confidence
embedding_path
domain_metadata
```

Prediction files must be immutable and hashable.

37. Classification metrics

Implement without hidden averaging assumptions:

- Accuracy.
- Top-k accuracy.
- Macro, micro, and weighted F1.
- Balanced accuracy.
- Mean per-class recall.
- Per-class precision and recall.
- Confusion matrix.
- Negative log likelihood.
- Brier score.
- Expected calibration error.
- Adaptive calibration error.
- Reliability bins.

38. Segmentation metrics

Implement:

- Pixel accuracy.
- Mean IoU.
- Frequency-weighted IoU.
- Per-class IoU.
- Dice.
- Boundary F1.
- Selective mIoU under uncertainty rejection.

39. Detection and instance metrics

Use optional `pycocotools` integration when installed.

Support:

- COCO AP.
- AP50.
- AP75.
- AP small, medium, and large.
- AR.
- Per-class AP.
- Count MAE from detections.

Provide clear dependency errors rather than silently replacing COCO metrics.

40. Domain metrics

Implement grouped evaluation by arbitrary manifest columns.

Report:

- Mean domain score.
- Worst-domain score.
- Domain standard deviation.
- In-domain score.
- Out-of-domain score.
- Absolute OOD drop.
- Relative OOD drop.
- Performance retention.

41. Label-efficiency analysis

Implement:

1. Deterministic label subset creation.
2. Fractions of 1, 5, 10, 25, 50, and 100 percent.
3. Fixed examples-per-class subsets.
4. Five subset seeds.
5. Shared subsets across models.
6. Area under the label-efficiency curve.
7. Labels required to match a reference score.
8. Performance retention at each fraction.
9. Split manifest hashes.

42. Calibration and selective prediction

Implement:

1. Temperature scaling.
2. Validation-only threshold selection.
3. Risk-coverage curves.
4. Accuracy at configured coverage.
5. Macro F1 at configured coverage.
6. Selective mIoU.
7. Calibration before and after scaling.
8. Serialization of calibration parameters.

43. Agricultural corruption suite

Create configurable tensor or PIL transformations for:

```
defocus blur
motion blur
JPEG compression
sensor noise
underexposure
overexposure
white-balance shift
lens flare approximation
fog
rain
dust
water droplets
soil splatter
partial occlusion
wind-like motion
```

Requirements:

1. Five severity levels.
2. Fixed random seeds.
3. No permanent modification of datasets.
4. Original sample ID preservation.
5. Mean corruption error.
6. Relative corruption error.
7. Worst corruption score.
8. Calibration under corruption.
9. Unit tests using small synthetic images.

44. Bootstrap and hypothesis tests

Implement:

1. IID bootstrap.
2. Cluster bootstrap by plant, field, farm, country, or sequence.
3. Paired bootstrap model comparison.
4. Wilcoxon signed-rank test.
5. Holm correction.
6. Effect-size reporting.
7. Confidence intervals.
8. Deterministic seeded resampling.
9. Handling of missing model-dataset combinations.
10. Structured JSON and CSV output.

Use SciPy as an optional benchmark dependency where needed.

45. Result aggregation

Implement aggregate tables for:

- Average rank.
- Task-family win count.
- Worst-domain score.
- Label-efficiency area.
- Mean global-task score.
- Mean dense-task score.
- Compute-normalized score.

Do not combine unrelated metrics without explicit normalization.

Store both raw metrics and normalized summary scores.

Part G. Experiment orchestration

46. Experiment matrix generator

Create an experiment matrix tool.

It should generate configs for:

Initialization

```
random
imagenet
```

```
mae
dinov2
dinov3
```

Objectives

```
mim
dino
dino_ibot
dino_ibot_morphology
dino_ibot_morphology_gram
sequential_mim_dino
```

Mask ratios

```
0.40
0.60
0.75
```

Sampling

```
proportional
sqrt_balanced
uniform_source
```

Model scale

```
S
B
L
```

Label fractions

```
0.01
0.05
0.10
0.25
0.50
1.00
```

Adaptation

```
linear
dense_linear
lora
decoder_only
full
```

Requirements:

1. Deterministic experiment IDs.
2. Collision detection.
3. Config inheritance.
4. Seed expansion.
5. Dry-run command generation.
6. Resource tag support.
7. Dependency graph support.
8. No execution unless explicitly requested.
9. CSV and JSON matrix output.

47. Benchmark runner

Implement a runner that can:

1. Validate all configurations.
2. Resolve model and dataset registries.
3. Create output directories.
4. Train or evaluate one task.
5. Resume safely.
6. Save predictions.
7. Save metrics.
8. Save checkpoint hashes.
9. Record split hashes.
10. Record trainable parameter counts.
11. Record runtime and peak memory.
12. Support dry-run.
13. Support CPU test doubles.
14. Refuse to use the test split during hyperparameter search.

48. Compute tracking

Add:

- Wall-clock time.
- Optimizer-step count.
- Samples seen.

- Tokens seen.
- Effective batch size.
- Parameter count.
- Trainable parameter count.
- Peak CPU memory when available.
- Peak CUDA memory when available.
- Device name.
- Device count.
- Throughput.
- Optional energy tracking when a supported library is installed.

The absence of optional energy tooling must not break training.

49. Reproducibility bundle

Implement `export-repro-bundle`.

It should collect:

```
Git commit
Git dirty state
resolved config
run manifest
dataset manifest hash
split hash
checkpoint hash
prediction hash
metrics
command log
environment
package versions
seed
compute record
```

Do not copy large checkpoints unless explicitly requested.

Part H. Tests

50. Required new test areas

Create focused tests for every new module.

Suggested files include:

```
tests/test_distributed_runtime.py
tests/test_gradient_accumulation.py
tests/test_step_schedules.py
tests/test_manifest_schema.py
tests/test_source_balanced_sampler.py
tests/test_split_registry.py
tests/test_exact_hashing.py
tests/test_perceptual_hashing.py
tests/test_contamination.py
tests/test_ibot_loss.py
tests/test_random_block_mask.py
tests/test_foreground_mask.py
tests/test_boundary_mask.py
tests/test_mixed_mask.py
tests/test_mask_curriculum.py
tests/test_morphology_mim.py
tests/test_gram_anchor.py
tests/test_joint_ssl.py
tests/test_pipeline_stages.py
tests/test_high_resolution.py
tests/test_benchmark_registry.py
tests/test_benchmark_datasets.py
tests/test_encoder_interface.py
tests/test_lora.py
tests/test_linear_probe.py
tests/test_dense_probe.py
tests/test_upernet_head.py
tests/test_vit_feature_pyramid.py
tests/test_counting.py
tests/test_retrieval.py
tests/test_open_set.py
tests/test_calibration.py
tests/test_corruptions.py
tests/test_domain_metrics.py
tests/test_label_efficiency.py
tests/test_bootstrap.py
tests/test_result_aggregation.py
tests/test_experiment_matrix.py
tests/test_repro_bundle.py
tests/test_new_cli_commands.py
```

Tests must:

1. Use synthetic tensors and temporary files.
2. Complete quickly on CPU.
3. Avoid real `timmm` downloads.

4. Avoid real benchmark data.
5. Avoid real distributed launch.
6. Avoid more than two tiny optimization steps.
7. Assert shapes, gradients, state changes, determinism, and error behavior.
8. Validate checkpoint round trips.
9. Validate backward compatibility.
10. Validate CLI help.

51. Commands you may run

Run only targeted commands such as:

```
python -m pytest tests/test_ibot_loss.py -q
python -m pytest tests/test_joint_ssl.py tests/test_gradient_accumulation.py -q
python -m pytest tests/test_benchmark_registry.py -q
python -m ruff check path/to/modified/files
```

Do not run:

```
python -m pytest
python -m pytest -q
python -m ag_foundation train-mim
python -m ag_foundation train-dino
python -m ag_foundation train-joint
python scripts/analyze_pretraining_dataset.py
```

Do not inspect the actual dataset to prove that code works.

Part I. Configuration templates

52. Add complete templates

Create publication-oriented templates under:

```
configs/pretraining/
configs/pipelines/
configs/downstream/
configs/ablations/
configs/benchmarks/
configs/robustness/
configs/statistics/
```


Include templates for:

1. ViT S recipe screening.
2. ViT B joint pretraining.
3. DINO baseline.
4. MIM baseline.
5. DINO plus iBOT.
6. Morphology masking.
7. Gram anchoring.
8. Sequential MIM then DINO.
9. High-resolution refinement.
10. Source sampling ablations.
11. Data-scale ablations.
12. Model-scale ablations.
13. Linear probing.
14. Dense probing.
15. LoRA.
16. Full fine-tuning.
17. Detection.
18. Semantic segmentation.
19. Instance segmentation.
20. Counting.
21. Retrieval.
22. Open-set evaluation.
23. Domain-shift evaluation.
24. Corruption evaluation.
25. Statistical aggregation.

Use placeholder paths such as:

```
/path/to/manifest.csv  
/path/to/checkpoint.pt
```

Do not use the user's local filesystem paths.

Part J. Documentation

53. Update documentation with implemented behavior

Update:

```
README.md  
docs/architecture.md  
docs/configuration.md
```

```
docs/data-formats.md
docs/training.md
docs/experiment-tracking.md
docs/reproducibility.md
docs/testing.md
docs/research-roadmap.md
docs/implementation-map.md
docs/troubleshooting.md
```

Document:

1. New CLI commands.
2. Joint SSL objective.
3. iBOT behavior.
4. Morphology masks.
5. Gram anchoring.
6. Distributed launch.
7. Gradient accumulation.
8. Source-balanced sampling.
9. Benchmark framework.
10. Adaptation modes.
11. Metrics.
12. Dry-run behavior.
13. Optional dependencies.
14. Config examples.
15. Precise scientific naming.
16. Features that require real data and have not been scientifically validated.

Do not claim benchmark superiority or successful scientific validation.

Part K. Packaging and CI

54. Optional dependency groups

Update package metadata with sensible optional groups such as:

```
distributed
benchmark
detection
segmentation
statistics
all
```

Keep the basic package importable without all optional dependencies.

55. CI design

Update CI definitions but do not execute the full CI locally.

CI should include:

1. Core unit tests.
2. Benchmark unit tests with synthetic data.
3. Optional dependency import checks.
4. Linux, macOS, and Windows compatibility where practical.
5. Python versions already supported by the project.
6. No pretrained downloads.
7. No actual training.
8. No real dataset access.
9. Ruff.
10. CLI help tests.

Part L. Backward compatibility

56. Preserve existing behavior

Confirm through focused tests that:

1. Existing MIM configs still parse.
2. Existing DINO configs still parse.
3. Existing MIM checkpoints still load.
4. Existing DINO checkpoints still load.
5. Existing RGB identity adapters behave identically.
6. Existing multispectral one-by-one adapters remain supported.
7. Existing command logging remains append-only.
8. Existing run manifests remain readable.
9. Existing resume semantics remain unchanged.
10. Existing demo data generation remains available.
11. Existing public imports are preserved or migrated with compatibility exports.

Part M. Final internal audit

57. Audit the completed implementation

Before finishing:

1. Search modified production code for `TODO`, `FIXME`, `NotImplementedError`, and empty `pass` statements.
2. Verify every new CLI command has a registered handler.

3. Verify every new configuration key is documented.
4. Verify every new checkpoint state has save and restore logic.
5. Verify rank-zero artifact behavior.
6. Verify optional dependencies fail clearly.
7. Verify no real data path was inspected.
8. Verify no network call occurred.
9. Verify no full test suite or training command was run.
10. Run only the targeted tests for modified modules.
11. Run Ruff only on modified files.
12. Review `git diff` for accidental data files, generated checkpoints, caches, or machine-specific paths.

58. Final response format

Do not paste large code blocks in the final response.

The repository itself is the deliverable.

Return only:

1. A concise list of completed implementation areas.
2. A list of important new files.
3. The exact targeted test commands that were run.
4. The result of those targeted tests.
5. Tests intentionally not run because of the stated restrictions.
6. A statement confirming that no real data inspection, full training, pretrained download, full benchmark, or full test suite was performed.
7. Any genuine unresolved blocker that could not be solved without real data or unavailable external dependencies.

Do not describe planned future code. Do not claim completion unless all requested code paths are implemented and the targeted tests pass.